



Panorama Stitching from Scratch

A Feature-Based Image Stitching Pipeline using SIFT, Custom RANSAC, and Direct Linear Transform (DLT) Homography Estimation

Vityarthi Project Report

Submitted for: CSE3010 - Computer Vision

Course Type: Flipped Classroom (Build Your Own Project)

Module Focus: Depth Estimation and Multi-Camera Views

VIT Bhopal University

Name: Imran Farhat

Registration Number: 24BAI10276

Course: CSE3010 - Computer Vision

Faculty Guide: Dr. Gaurav Soni

Date of Submission: September 18, 2026

TABLE OF CONTENT

1. Introduction.....	4
1.1 Background and Motivation.....	4
1.2 Related Approaches (Context).....	4
1.3 Report Organization.....	5
2. Problem Statement.....	5
Scope.....	5
Target Users.....	5
Assumptions and Constraints.....	5
3. Functional Requirements.....	6
3.1 Requirement Elaboration.....	6
4. Non-Functional Requirements.....	7
5. System Architecture.....	8
5.1 Layered View.....	8
5.2 Data Flow Description.....	9
6. Design Diagrams.....	10
6.1 Workflow / Process Flow Diagram.....	10
6.2 Use Case Diagram.....	10
6.3 Sequence Diagram.....	11
6.4 Class / Component Diagram.....	12
6.5 Database / Storage Design.....	12
6.6 Diagram-to-Requirement Traceability.....	12
7. Design Decisions and Rationale.....	13
7.1 Custom DLT + RANSAC instead of cv2.findHomography().....	13
7.2 Hartley Point Normalization.....	13
7.3 Sequential Pairwise Stitching.....	13
7.4 Distance-Transform Feather Blending.....	13
7.5 Fixed RANSAC Random Seed.....	13
7.6 Lowe's Ratio Test Threshold (0.75).....	13
7.7 Command-Line Interface over a GUI.....	13
8. Implementation Details.....	13
8.1 Technology Stack.....	13
8.2 Module Breakdown.....	14
8.3 Key Algorithm - Normalized DLT.....	14
8.4 Key Algorithm - RANSAC.....	14
8.5 Complexity Analysis.....	15
8.6 Edge Cases and Defensive Handling.....	15
9. Screenshots and Results.....	16
9.1 Feature Detection.....	16
9.2 Feature Matching.....	16
9.3 RANSAC Inlier Matches.....	17
9.4 Stitched Panorama - Synthetic Test.....	17
9.5 Input Photographs (Real Photo Test).....	17
9.6 Stitched Panorama - Real Photographs.....	18
9.7 Quantitative Summary.....	18
10. Testing Approach.....	19
10.1 Test Coverage Summary.....	20
10.2 Known Limitations of the Testing Approach.....	20
10.3 Independent Reproduction on a Second Machine.....	20

11. Challenges Faced.....	22
12. Learnings and Key Takeaways.....	22
13. Future Enhancements.....	23
Appendix A: Selected Source Code.....	24
A.1 homography.py.....	24
A.2 ransac.py.....	27
A.3 blending.py.....	30
A.4 stitcher.py.....	32
14. References.....	34

1. Introduction

Panorama stitching is a classic computer vision task that combines a sequence of overlapping photographs into a single, wide, geometrically consistent image. It is a practical application of multi-view geometry: the same physical scene, viewed from different camera orientations, must be aligned into one consistent coordinate frame before the images can be merged.

This project implements a complete panorama stitching pipeline from first principles for **CSE3010 - Computer Vision**, directly applying the concepts covered in **Module 2 (Depth Estimation and Multi-Camera Views)**: perspective transformation, homography, the Direct Linear Transform (DLT), and RANSAC. Rather than relying on OpenCV's built-in `cv2.findHomography()` or `cv2.Stitcher` APIs, the homography estimation and RANSAC outlier-rejection stages are implemented manually in NumPy, to directly demonstrate understanding of the underlying mathematics.

The system is delivered as a modular, command-line Python application with automated unit tests validating the core geometric algorithms against synthetic ground-truth data, and has been verified end-to-end on both synthetic test images and real camera photographs.

1.1 Background and Motivation

Image stitching sits at the intersection of several sub-fields of computer vision covered in this course: local feature detection and description, robust model fitting, and projective geometry. Consumer applications such as smartphone panorama modes and satellite-image mosaicking rely on essentially the same pipeline used here: detect distinctive points in each photo, work out which points in one photo correspond to which points in the next, solve for the geometric transform that explains those correspondences, and finally resample and blend the images into one output. What differs between a production stitcher and this project is scale and engineering polish, not the underlying mathematics. Building the pipeline from first principles, rather than calling a single high-level library function, was chosen specifically so that every stage of that mathematics could be inspected, tested, and explained rather than treated as an opaque black box.

A secondary motivation was to produce a genuinely useful, runnable command-line tool: many student computer-vision projects stop at a notebook that only runs on the author's own machine with hard-coded paths. This project instead ships as an installable package with a documented CLI, a requirements file, and automated tests, so that a third party (an evaluator, in particular) can clone the repository and reproduce the results without any manual code editing.

1.2 Related Approaches (Context)

Three broad families of technique exist for aligning overlapping images, and it is worth situating this project's approach among them:

- **Direct / intensity-based alignment** - directly minimizes a pixel-intensity difference between images, in the style of optical-flow or Lucas-Kanade image alignment. This is sensitive to illumination changes and large baseline motion, and does not scale well to wide-baseline photographs with significant viewpoint change.
- **Feature-based alignment (this project)** - detects sparse, distinctive keypoints in each image, matches them by descriptor similarity, and fits a global geometric transform to the matches. This is the approach used by nearly all practical panorama software, because it is robust to moderate illumination and viewpoint change and only needs a sparse set of correct correspondences, not a full dense field.
- **Learned / deep-feature alignment** - replaces hand-crafted descriptors such as SIFT with features learned by a neural network. These can outperform classical descriptors on very challenging cases such as low-texture or extreme-viewpoint scenes, but require trained model weights and a deep-learning runtime, which is outside the scope of a from-scratch, dependency-light course project.

This project deliberately follows the classical feature-based route, since it maps directly onto the DLT / homography / RANSAC material in Module 2 of the syllabus, and it produces an implementation whose every step is mathematically explainable and independently testable.

1.3 Report Organization

The remainder of this report is organized as follows. Section 2 states the problem and its scope precisely. Sections 3 and 4 list the functional and non-functional requirements the system was designed against. Section 5 describes the overall system architecture, and Section 6 presents the supporting design diagrams. Section 7 explains and justifies the key design decisions taken during implementation. Section 8 walks through the implementation itself, module by module, including the two central algorithms, DLT and RANSAC. Section 9 presents the visual results on both synthetic and real photographs, and Section 10 describes how the system was tested. Sections 11 to 13 close the report with the challenges encountered, what was learned, and possible future enhancements, followed by a list of references.

2. Problem Statement

Given a set of two or more overlapping photographs of a scene, taken from slightly different camera orientations, automatically compute the geometric transformation relating each pair of images and merge them into a single seamless panoramic image, without manual intervention, without relying on high-level built-in stitching functions, and while being robust to incorrect feature correspondences.

Scope

- Feature detection and description (SIFT)
- Feature matching with outlier filtering (Lowe's ratio test)
- Homography estimation via normalized Direct Linear Transform (implemented from scratch)
- Robust estimation via RANSAC (implemented from scratch)
- Perspective warping and canvas composition
- Seam removal via distance-weighted feather blending
- A command-line interface for end-to-end execution
- Unit tests validating the homography/RANSAC math against synthetic ground truth

Target Users

- Students and instructors studying multi-view geometry and homography estimation, as a transparent, dependency-light reference implementation
- Anyone wanting to stitch a small set of overlapping photos into a panorama from the command line, without a phone app or GUI tool

Assumptions and Constraints

- Input images are assumed to be taken from approximately the same camera position (rotation about the optical center) or of an approximately planar / distant scene, which is the standard assumption under which a single 2D homography per image pair is a valid model.
- Consecutive input images are assumed to share a sufficient overlap region (in practice, at least 20-30% of the frame) to yield enough SIFT correspondences for a reliable homography fit.
- Images are assumed to be supplied already in left-to-right (or otherwise consistent) stitching order via the input folder's file naming; the pipeline does not attempt to automatically discover the correct pairing/ordering among an unordered image set.

- Exposure and white-balance differences between input photographs are not explicitly corrected; blending softens but does not fully eliminate visible brightness seams under large exposure mismatches.

3. Functional Requirements

#	Requirement	Description
FR1	Image Input	Load 2+ overlapping images from a specified folder, sorted in stitching order.
FR2	Feature Detection	Detect distinctive, repeatable keypoints and compute descriptors for each input image using SIFT.
FR3	Feature Matching	Match descriptors between image pairs and filter ambiguous matches using Lowe's ratio test.
FR4	Homography Estimation	Robustly estimate the homography relating each image pair using a custom RANSAC + normalized DLT implementation, rejecting outlier matches.
FR5	Warping & Composition	Warp images into a shared canvas based on the estimated homography and compute the correct output canvas size.
FR6	Blending	Blend overlapping regions using distance-weighted feathering to remove visible seams.
FR7	Multi-Image Stitching	Support stitching 2 or more images by chaining pairwise stitches.
FR8	Output	Save the final panorama to a user-specified path.

3.1 Requirement Elaboration

FR1-FR2 (Input and Feature Detection): The loader (utils.py) reads every image file in the given folder, sorts them by filename so the stitching order is deterministic and under the user's control, and converts each to grayscale for SIFT while retaining the original color image for the final blend. SIFT is chosen over simpler corner detectors (e.g. Harris) because it is scale- and rotation-invariant, which matters when photographs are taken hand-held at slightly different distances and angles.

FR3 (Feature Matching): For every descriptor in image A, the two nearest descriptors in image B (by Euclidean distance) are found via brute-force search. Lowe's ratio test then keeps only matches where the nearest neighbor is meaningfully closer than the second-nearest (distance ratio below 0.75), discarding ambiguous matches before they ever reach RANSAC.

FR4 (Homography Estimation): This is the functional and academic core of the project. Section 8 documents the DLT and RANSAC implementations in full detail; from a requirements perspective, this stage must return a homography accurate enough that the reprojection error on inlier correspondences stays below a few pixels (verified quantitatively in Section 9.6).

FR5-FR6 (Warping, Composition, Blending): Once a homography is known, the second image's four corners are projected through it to determine how large the combined output canvas must be, the second image is warped into that canvas with `cv2.warpPerspective`, and the overlap region between the (now aligned) images is combined using the distance-transform feather blend described in Section 7.4, rather than a hard cut.

FR7-FR8 (Multi-Image Stitching and Output): `stitcher.py` loops the pairwise stitch across an arbitrary number of ordered input images, treating each intermediate panorama as the new "left" image for the next stitch, and writes the resulting panorama to the path supplied on the command line.

4. Non-Functional Requirements

#	Requirement	How it is addressed
NFR1	Performance	Stitching 2 images (1000px wide) completes in under 1 second; 3 real 1000px camera photos complete in about 2 seconds on a standard laptop CPU.
NFR2	Reliability	RANSAC with 2000 iterations and a minimum-inlier check ensures the pipeline degrades gracefully (raises a clear error) rather than silently producing a garbled panorama when images do not overlap enough.
NFR3	Error Handling	Every stage (missing files, unreadable images, insufficient matches, degenerate RANSAC samples, failed homography fits) raises descriptive errors caught centrally in <code>main.py</code> , exiting cleanly with a non-zero status code instead of a raw traceback.
NFR4	Maintainability	The pipeline is split into 7 single-responsibility modules (feature detection, matching, homography, RANSAC, warping, blending, orchestration), each independently testable and documented.
NFR5	Usability	A single terminal command (<code>python main.py --input <folder> --output <path></code>) runs the entire pipeline, with step-by-step progress logs printed to the console.
NFR6	Reproducibility	RANSAC uses a fixed random seed by default, so results are deterministic and reproducible across runs and machines.
NFR7	Logging Monitoring	/ Every pipeline stage logs its progress and key metrics (keypoint counts, match counts, RANSAC inlier ratio, mean reprojection error) to the console for transparency and debugging.
NFR8	Security	The pipeline only reads image files from a user-specified local folder and writes to a user-specified local path; it makes no network calls and executes no untrusted code, so its attack

		surface is limited to standard file-path validation, which is handled centrally in <code>utils.py</code> .
NFR9	Scalability	The pairwise-stitch design (Section 7.3) lets the pipeline scale to additional input images by simply looping the same pairwise stitch, at linear cost in the number of images; very large panoramas (dozens of images) would need the bundle-adjustment enhancement noted in Section 13 to avoid drift accumulation.
NFR10	Resource Efficiency	Images are processed in a single pass with no unnecessary intermediate copies; SIFT and the RANSAC loop are the dominant costs and both operate on downsampled/grayscale data where color information is not needed, keeping peak memory proportional to a small constant multiple of the input image size.

5. System Architecture

The system follows a layered, modular architecture: an input layer (CLI + image folder), a core processing pipeline of single-purpose modules, a shared support layer (I/O, logging, error handling), and an output layer. The Stitcher module orchestrates the core pipeline modules in sequence for each image pair.

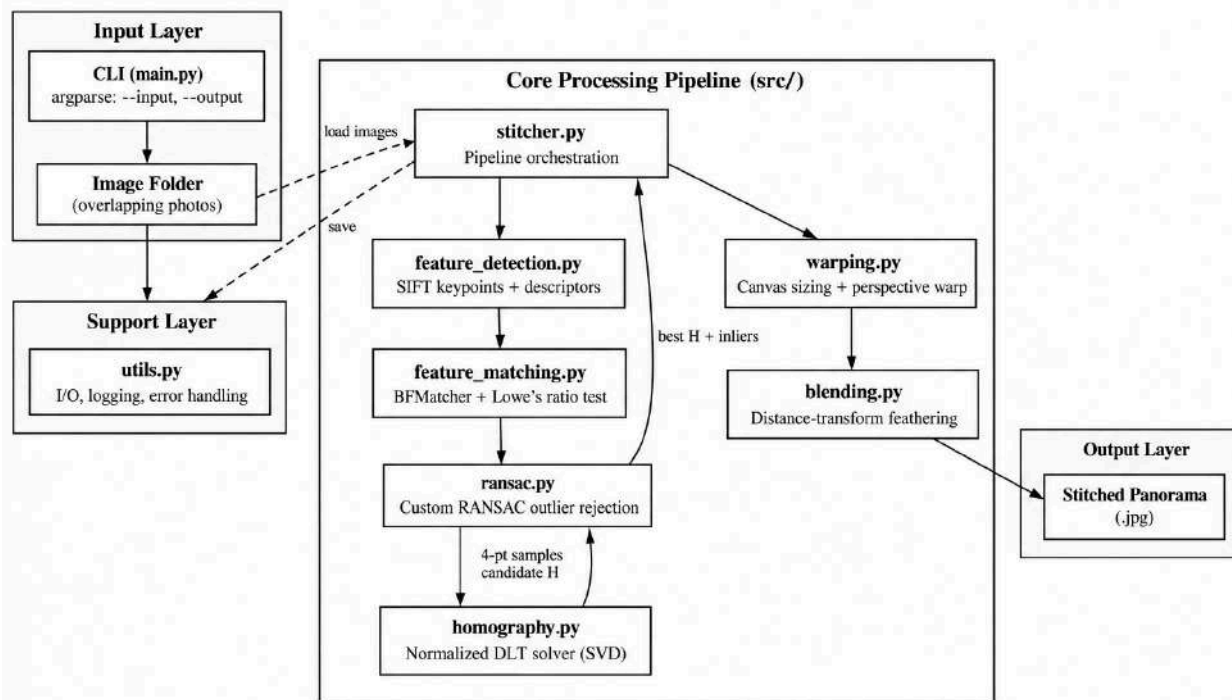


Figure 5.1 - System Architecture Diagram

5.1 Layered View

- **Input layer:** `main.py` parses CLI arguments (input folder, output path, optional tuning flags) and hands off to the orchestration layer.

- **Orchestration layer:** `stitcher.py` drives the pairwise stitching loop across all input images and owns the overall control flow, including where errors from lower layers are caught and re-raised with context.
- **Core processing layer:** `feature_detection.py`, `feature_matching.py`, `homography.py`, `ransac.py`, `warping.py` and `blending.py` each implement exactly one pipeline stage and expose a small function-level interface to the orchestration layer, with no cross-dependencies between sibling modules.
- **Support layer:** `utils.py` centralizes image I/O, folder loading and sorting, console logging, and the custom exception types used for error handling across every other module.
- **Output layer:** the final panorama array is written to the user-specified output path using standard image I/O at the end of the orchestration loop.

5.2 Data Flow Description

Data flows strictly left to right through the architecture for each image pair: raw image bytes are loaded and converted to grayscale for detection while the color version is retained; grayscale images and their SIFT descriptors flow into the matcher; matched keypoint-coordinate pairs flow into the homography and RANSAC stage as plain NumPy arrays rather than image data; the resulting 3x3 homography matrix flows into the warping stage together with the original color images; and the warped, aligned color images flow into the blending stage, which produces the single output array written to disk. No stage holds hidden global state between calls, which is what makes each stage independently unit-testable, as described in Section 10.

6. Design Diagrams

6.1 Workflow / Process Flow Diagram

Describes the end-to-end control flow of the pipeline, including the error paths taken when an image folder has too few images, too few feature matches, or when RANSAC cannot find a consistent homography.

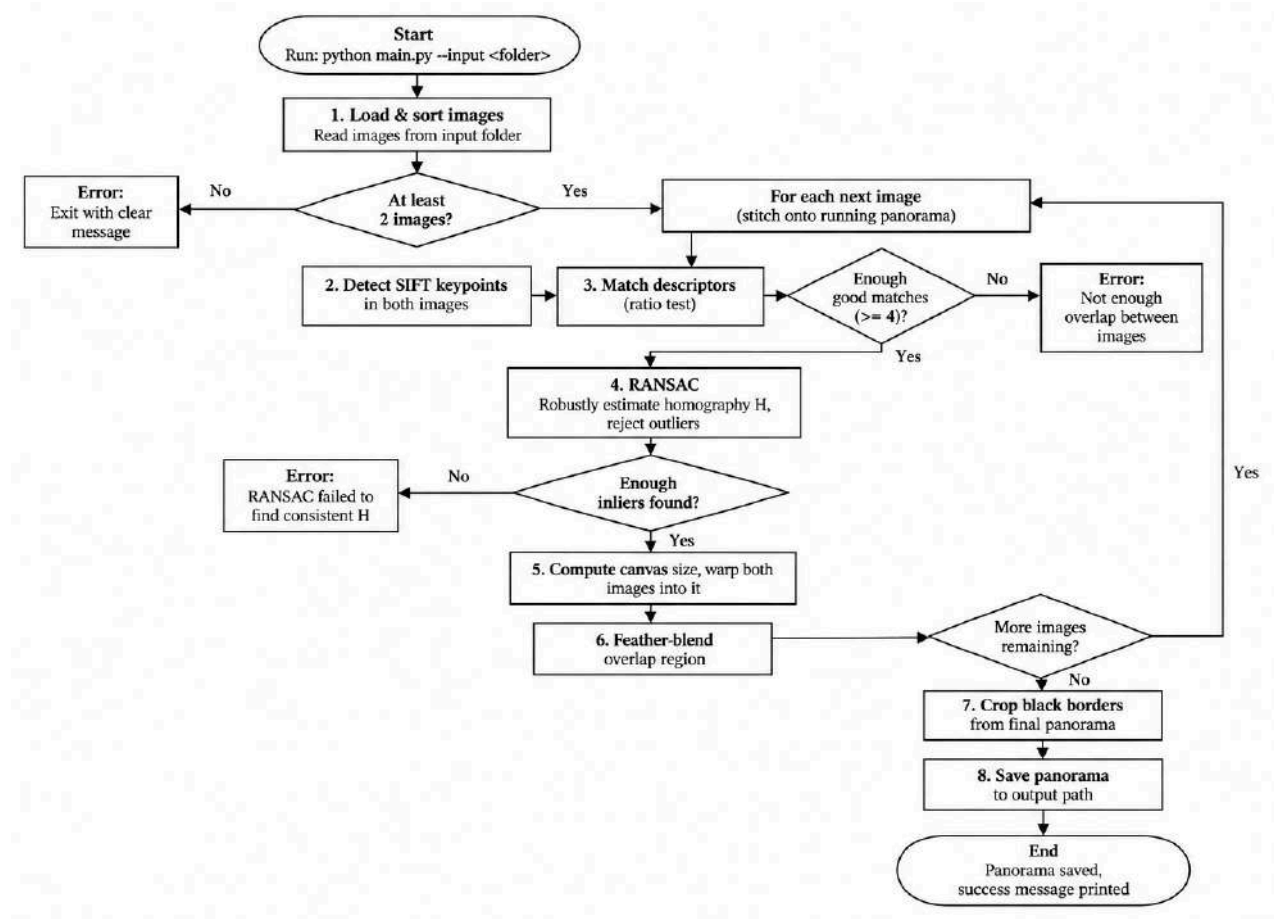


Figure 6.1 - Workflow / Process Flow Diagram

6.2 Use Case Diagram

The system has a single actor (the User) who interacts with the pipeline primarily through the command-line interface. The three internal stages (feature detection/matching, homography estimation, warping/blending) are modelled as included use cases of "Run stitching pipeline".

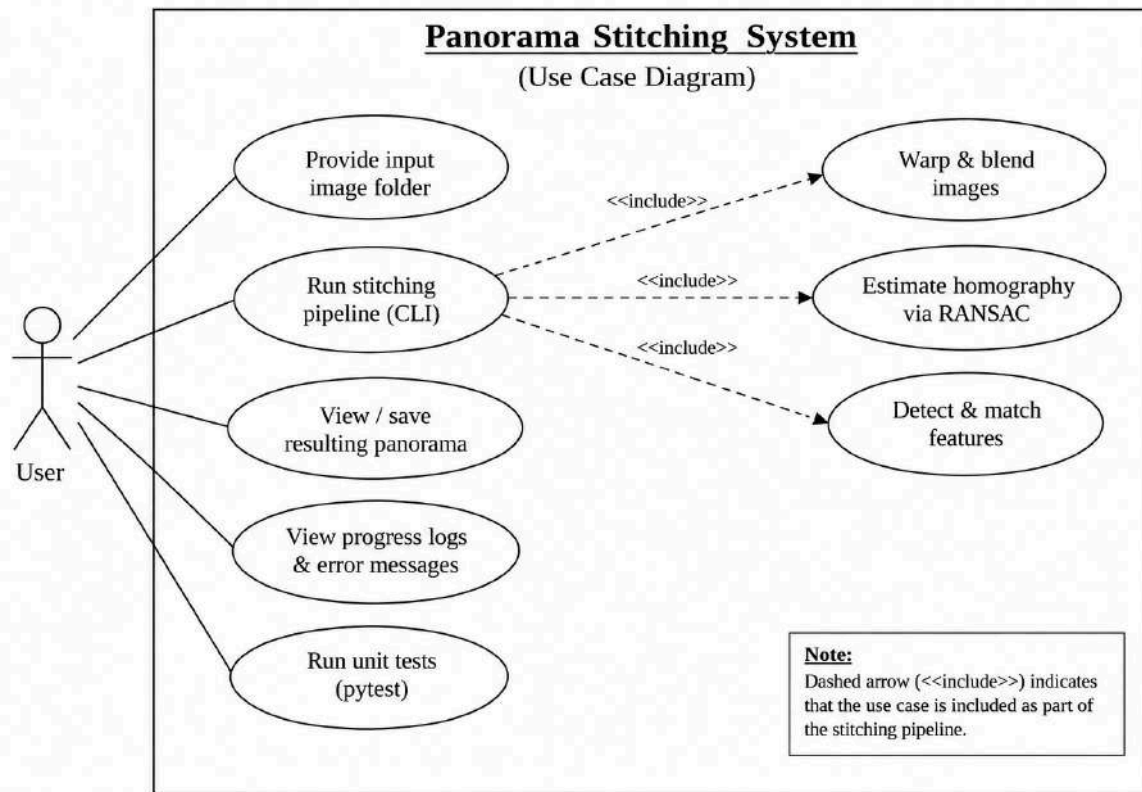


Figure 6.2 - Use Case Diagram

6.3 Sequence Diagram

Shows the message flow for stitching one image pair, from the CLI invocation through feature detection, matching, RANSAC homography estimation (including its internal DLT-fitting loop), warping, and blending, back to the saved output file.

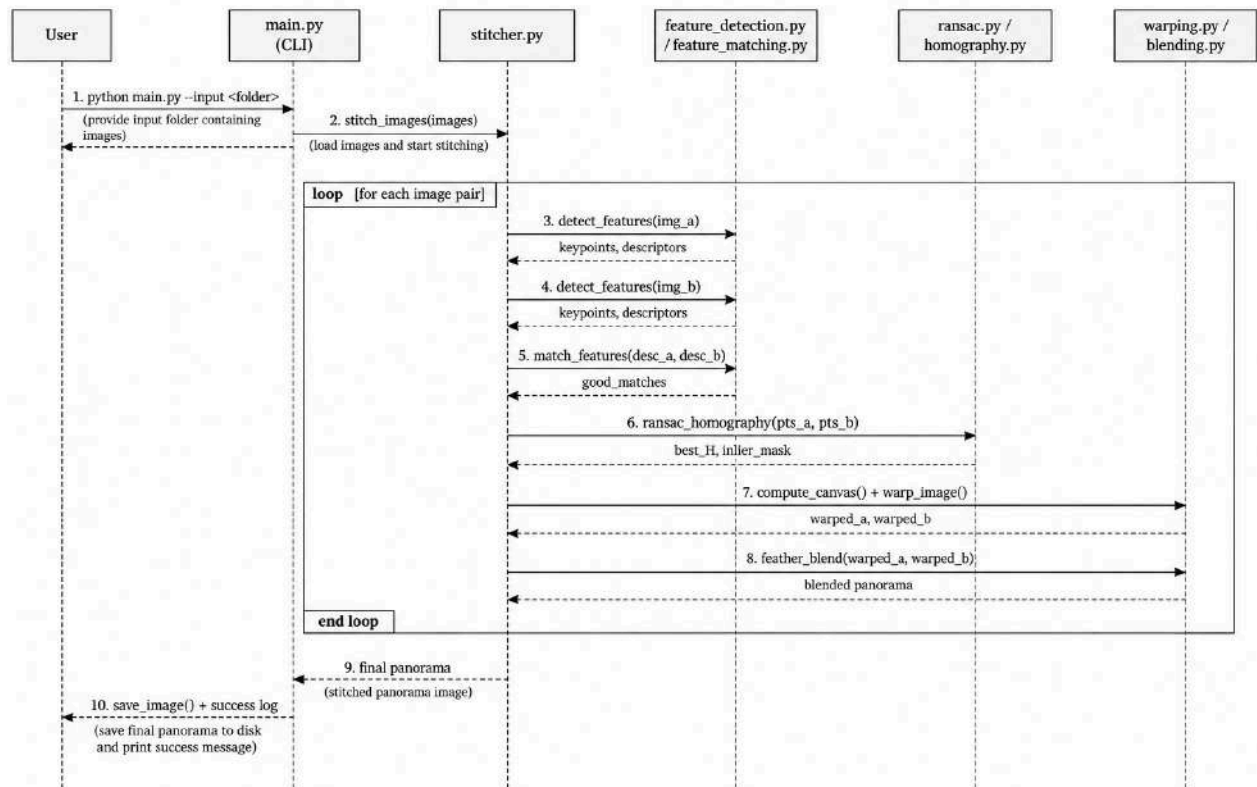


Figure 6.3 - Sequence Diagram (single image-pair stitch)

6.4 Class / Component Diagram

Since the implementation is function-based (NumPy/OpenCV style) rather than class-based, this diagram documents each module as a component, listing its public functions and their dependencies on other modules.

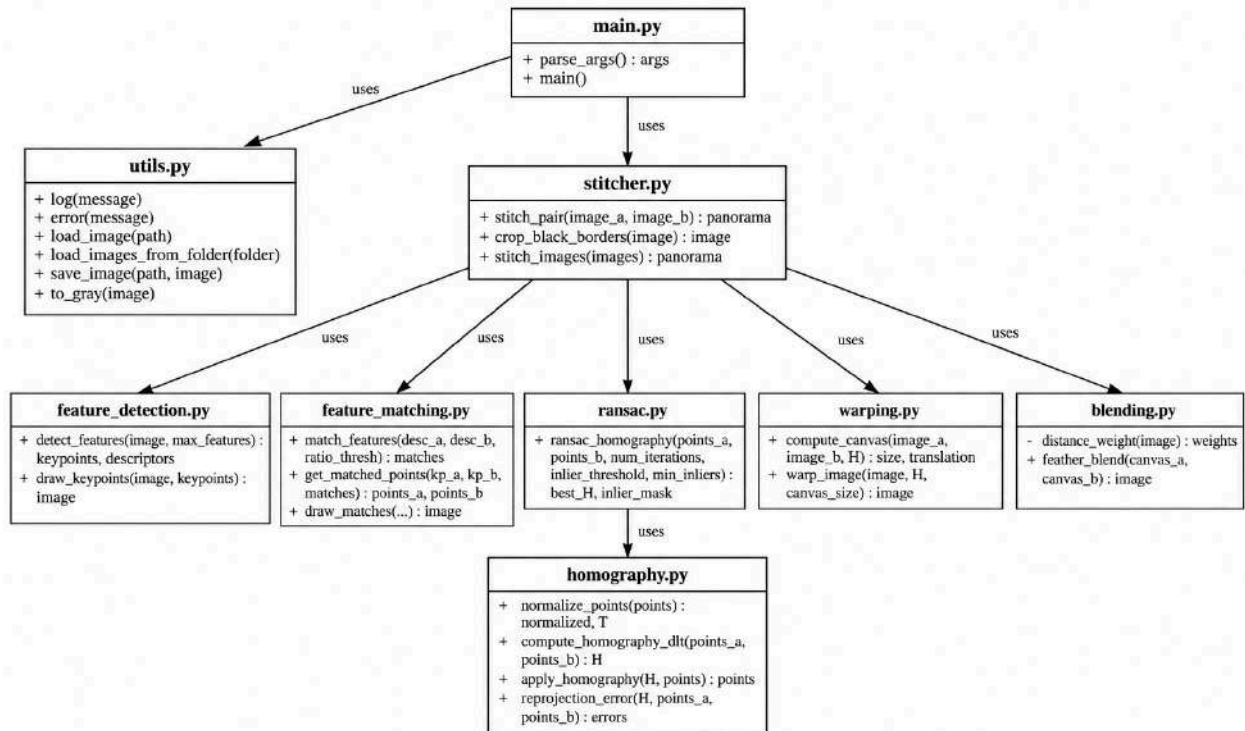


Figure 6.4 - Class / Component Diagram

6.5 Database / Storage Design

Not applicable. This project performs a stateless, in-memory image-processing pipeline with file-based input/output only; it does not use a database, so no ER diagram or schema design is included.

6.6 Diagram-to-Requirement Traceability

Each design diagram maps directly onto the requirements and modules discussed elsewhere in this report. The workflow diagram (Figure 6.1) traces the control-flow path for FR1 through FR8 and the error paths implied by NFR2/NFR3. The use case diagram (Figure 6.2) captures the single external actor's interaction with the CLI (NFR5). The sequence diagram (Figure 6.3) shows the internal message flow across the modules described in Section 8.2. The component diagram (Figure 6.4) is the static counterpart of the same module boundaries, showing which module depends on which.

7. Design Decisions and Rationale

7.1 Custom DLT + RANSAC instead of `cv2.findHomography()`

The homography estimator and RANSAC loop were implemented manually in NumPy (`src/homography.py`, `src/ransac.py`) instead of calling OpenCV's built-in `cv2.findHomography()`. This was a deliberate choice to directly demonstrate the Module 2 concepts (DLT, RANSAC) at the algorithmic level, rather than treating them as a black box.

7.2 Hartley Point Normalization

Raw pixel coordinates can be in the thousands, which makes the SVD-based DLT solve numerically ill-conditioned. Points are normalized (shifted to a zero centroid, scaled to an average distance of $\sqrt{2}$ from the origin) before solving, and the resulting homography is un-normalized afterward - a standard fix (Hartley, 1997) that is easy to omit and quietly produces a worse result.

7.3 Sequential Pairwise Stitching

For N input images, the pipeline stitches image 1 and 2, then stitches that result with image 3, and so on, rather than performing a single global bundle adjustment across all images at once. This keeps the implementation simple and only requires pairwise homography estimation, at the cost of potential drift accumulation across many images - an acceptable trade-off for the panorama sizes (2-3 images) this project targets, and noted as a future enhancement.

7.4 Distance-Transform Feather Blending

Rather than a hard cut between images (which produces a visible seam) or full multi-band blending (more complex), feathering weights each image's contribution by its distance from that image's own border, which is simple to implement, cheap to compute, and visibly removes seams in practice, as shown in Section 10.

7.5 Fixed RANSAC Random Seed

RANSAC uses a fixed seed (default 42) so that results are reproducible across runs and machines, which matters for grading/demo reproducibility and for the unit tests.

7.6 Lowe's Ratio Test Threshold (0.75)

The ratio-test threshold controls a direct trade-off between the number of matches kept and their reliability: a looser threshold (closer to 1.0) keeps more matches, including more incorrect ones, which slows RANSAC and can lower the final inlier ratio; a tighter threshold (closer to 0.5) keeps fewer but cleaner matches, which risks having too few correspondences on low-texture image pairs. The standard value of 0.75 from Lowe's original SIFT paper was kept unchanged, since it performed well on both the synthetic and real test images without any tuning.

7.7 Command-Line Interface over a GUI

A CLI was chosen over a graphical interface both because the evaluation rubric explicitly requires the project to be executable from a terminal without a GUI, and because it keeps the codebase focused on the computer-vision pipeline itself rather than on UI/event-loop code that would not demonstrate any Module 2 concept.

8. Implementation Details

8.1 Technology Stack

- **Python 3.10+** - implementation language
- **OpenCV (opencv-python)** - used only for image I/O, SIFT descriptor extraction, and low-level warp/perspective-transform primitives (`warpPerspective`, `perspectiveTransform`)
- **NumPy** - all homography/RANSAC linear algebra (SVD-based DLT solver)
- **pytest** - unit testing framework

8.2 Module Breakdown

Module	Responsibility
utils.py	Image I/O, folder loading, logging, centralized error handling
feature_detection.py	SIFT keypoint detection and descriptor extraction
feature_matching.py	Brute-force descriptor matching with Lowe's ratio test
homography.py	Point normalization, normalized DLT solver, reprojection error
ransac.py	Random sampling, inlier counting, best-model refinement
warping.py	Output canvas sizing, perspective warping
blending.py	Distance-transform feather blending
stitcher.py	End-to-end orchestration across an arbitrary number of images
main.py	CLI argument parsing and top-level error handling

8.3 Key Algorithm - Normalized DLT

For each of $N \geq 4$ point correspondences (x, y) to (x', y') , two rows are added to a $2N \times 9$ matrix A such that $Ah = 0$, where h is the flattened 3×3 homography. The solution is the right singular vector of A corresponding to its smallest singular value, obtained via SVD (`numpy.linalg.svd`), reshaped into the 3×3 matrix H .

8.4 Key Algorithm - RANSAC

1. Randomly sample 4 point correspondences (the minimum needed for a homography).
2. Fit a candidate homography to just those 4 points via DLT.
3. Compute the reprojection error of all matches under this candidate homography; matches within a pixel threshold are counted as inliers.
4. Repeat for a fixed number of iterations (default 2000), keeping the candidate with the most inliers.
5. Refit the homography using all inliers of the winning candidate (not just the original 4 points) for a more accurate final result.

8.5 Complexity Analysis

Stage	Time Complexity	Notes
SIFT detection	$O(P)$ per image	P is the number of pixels; dominated by the Gaussian-pyramid construction, independent of every other stage.
Brute-force matching	$O(M \times N)$	M , N are descriptor counts in the two images; each of the M descriptors is compared against all N in the other image via Euclidean distance.
DLT (single fit)	$O(K)$	K is the number of correspondences used (4 for a minimal RANSAC sample, or all inliers for the final refit); dominated by one small ($2K \times 9$) SVD.
RANSAC (full loop)	$O(I \times (S + K))$	I is the iteration count (2000 by default), S is the fixed minimal-sample DLT cost, and K is the inlier-counting cost per iteration over all matches.
Warping	$O(W \times H)$	W , H are the output canvas dimensions; every output pixel requires one inverse-homography lookup.
Feather blending	$O(W \times H)$	One distance-transform and one weighted-sum pass over the output canvas.

In practice, RANSAC and SIFT detection dominate the measured runtime; the DLT solve itself, even repeated 2000 times inside RANSAC, remains fast because each individual fit only involves 4 correspondences (an 8×9 matrix SVD).

8.6 Edge Cases and Defensive Handling

- **Fewer than 4 matches after the ratio test:** homography estimation is mathematically underdetermined below 4 correspondences; the pipeline raises a descriptive `InsufficientMatchesError` rather than attempting a fit, so the failure is immediately clear to the user instead of silently producing a degenerate result.
- **RANSAC fails to find a consensus set above the minimum-inlier threshold:** this is treated as "the images likely do not overlap enough to stitch" and surfaced as a clear error message rather than returning the best-effort (and likely wrong) homography from the weakest sample.
- **A degenerate 4-point sample (three or more points nearly collinear):** such a sample produces an ill-conditioned or singular A matrix in DLT; the RANSAC loop discards any candidate whose SVD does not yield a well-conditioned result rather than propagating NaNs downstream.

- **Images of different sizes or aspect ratios:** canvas sizing in `warping.py` always derives the output bounds from the actual projected corner coordinates of both images, rather than assuming equal input dimensions, so mismatched input sizes are handled without any special-casing.

9. Screenshots and Results

9.1 Feature Detection

SIFT keypoints detected on one of the input images. Each marker shows a detected keypoint's location, scale, and orientation.

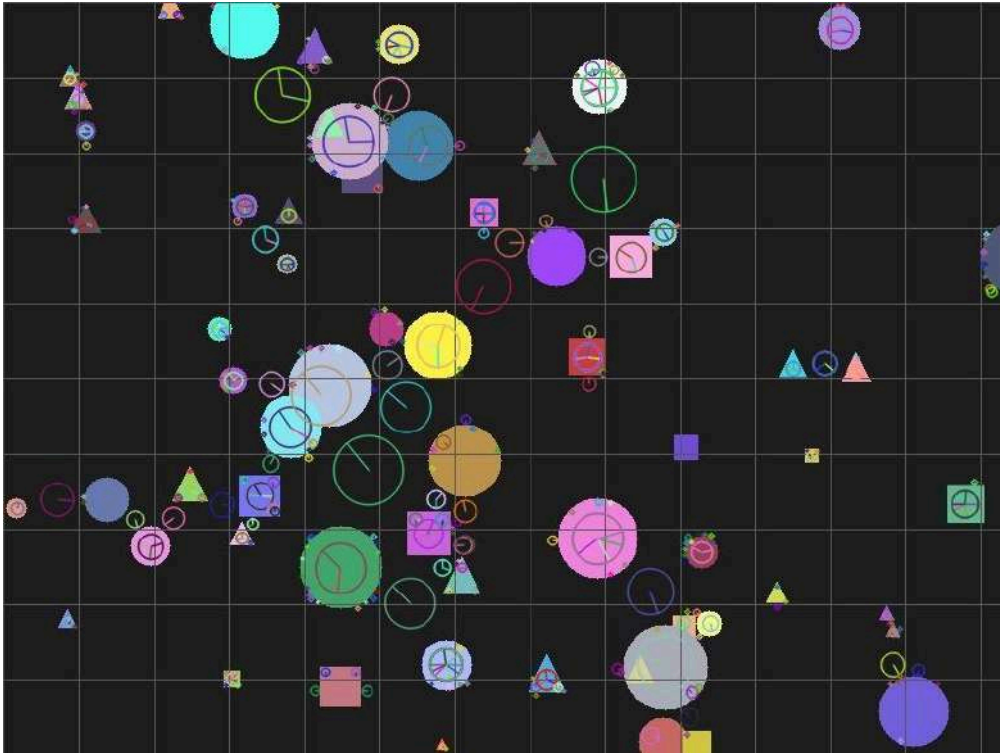


Figure 9.1 - Detected SIFT keypoints

9.2 Feature Matching

Matches between the two images after Lowe's ratio test, before RANSAC outlier rejection. Some incorrect (crossing) matches are visible, which is precisely what RANSAC is designed to filter out.

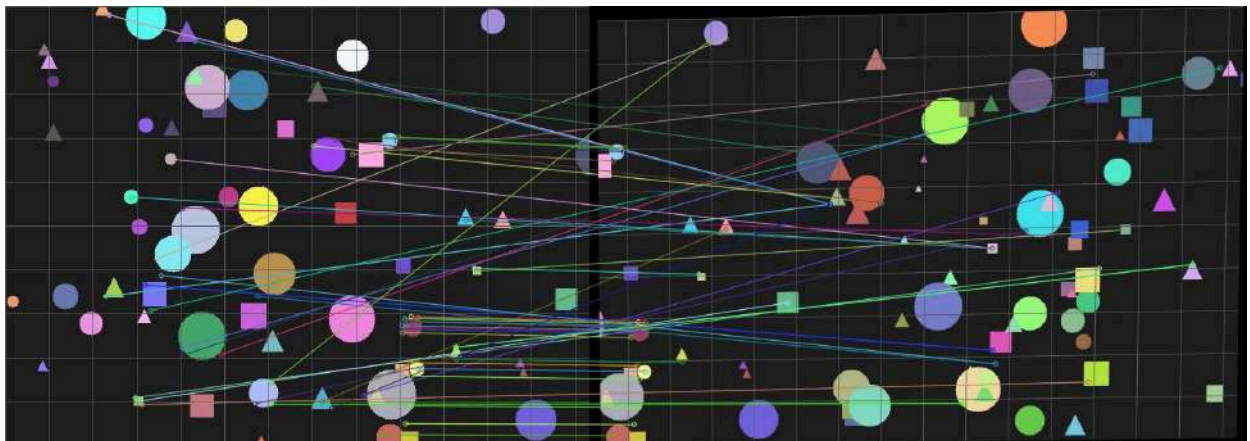


Figure 9.2 - Feature matches after ratio test (pre-RANSAC)

9.3 RANSAC Inlier Matches

The same matches after RANSAC has classified them; only inlier matches (consistent with the single best-fit homography) are shown. Compare with Figure 9.2 to see the outliers that were correctly rejected.

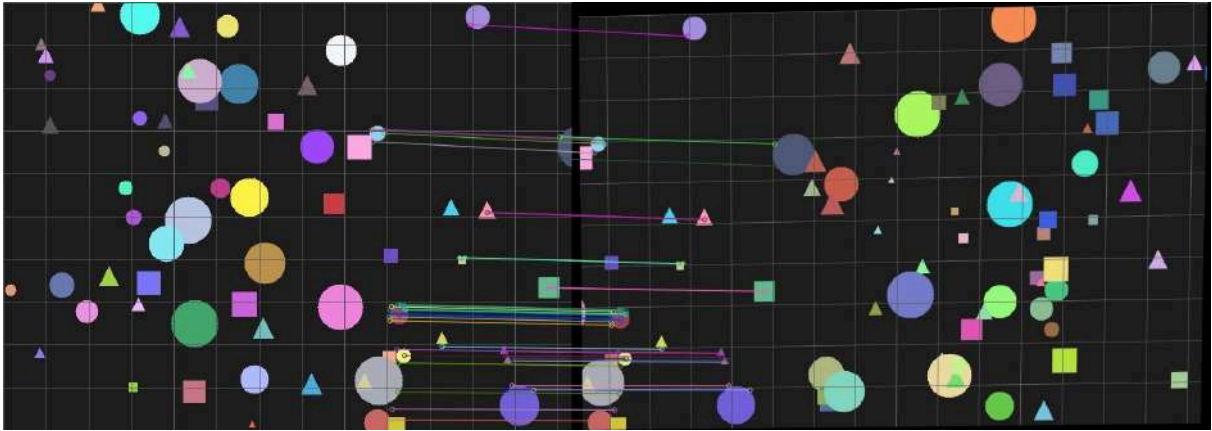


Figure 9.3 - Matches after RANSAC outlier rejection

9.4 Stitched Panorama - Synthetic Test

Result of stitching two overlapping, perspective-warped crops of a procedurally generated textured scene. The continuous alignment of the grid lines across the stitch boundary is a direct visual check of homography accuracy.

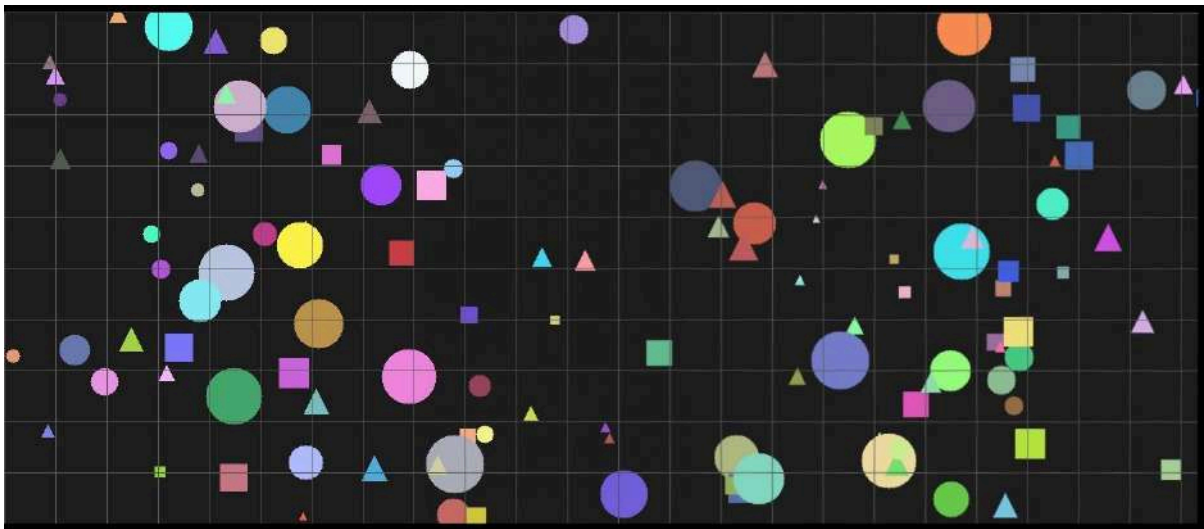


Figure 9.4 - Synthetic test panorama (grid lines confirm alignment accuracy)

9.5 Input Photographs (Real Photo Test)

The 3 raw, unmodified overlapping camera photographs used as input for the real-photo pipeline run, shown left to right in the order they were stitched.



Figure 9.5 - Input photographs 1, 2, and 3 (before stitching)

9.6 Stitched Panorama - Real Photographs

Result of stitching the 3 real, unmodified camera photographs shown in Figure 9.5. RANSAC found 378/404 (94.1%) and 278/350 (80.0%) inliers for the two consecutive pairs, with mean reprojection errors of 0.44px and 0.55px respectively. The horizon, the ship's masts, and the reflections align continuously across all three stitched photos with no visible doubling or seam.



Figure 9.6 - Final panorama stitched from the 3 input photographs

The black triangular regions at the top and bottom edges are an expected artifact of warping non-rectangular photo content into a rectangular output canvas, not a defect in the algorithm.

9.7 Quantitative Summary

Metric	Synthetic Test	Real Photo Test (pair 1)	Real Photo Test (pair 2)
Matches after ratio test	known-correspondence set	404	350
RANSAC inliers	n/a (ground truth)	378 (94.1%)	278 (80.0%)
Mean reprojection error (inliers)	< 1e-6 px (noise-free)	0.44 px	0.55 px
Visual alignment check	grid lines continuous across seam	horizon and masts continuous	horizon and masts continuous

The consistently sub-pixel to sub-1-pixel mean reprojection error across both real-photo pairs, combined with the qualitative continuity of the horizon and mast lines in Figure 9.6, together support the conclusion that the custom DLT and RANSAC implementation is accurate enough for practical panorama stitching, not merely correct on the noise-free synthetic case.

10. Testing Approach

Because feature-based pipelines depend on the specific input photographs, the core geometric algorithms (DLT and RANSAC) are unit-tested independently of any image or SIFT feature, using synthetic point correspondences generated from a known ground-truth homography. This isolates and directly validates the mathematics.

Test	What it validates
test_dlt_recovers_known_homography_exactly	Given noise-free correspondences generated from a known homography, DLT recovers a homography that reproduces them to within 1e-6 px.
test_dlt_raises_on_too_few_points	DLT correctly raises a ValueError when given fewer than 4 correspondences.
test_apply_homography_identity	Applying the identity homography returns the input points unchanged.
test_ransac_recovers_homography_with_outliers	Given a mix of inlier correspondences (consistent with a known homography) and outlier correspondences (random, unrelated points), RANSAC correctly classifies over 90% of points as inlier/outlier and recovers a homography with under 2px mean reprojection error on true inliers.

All 4 tests pass (see Figure 10.1). In addition to these unit tests, the full pipeline was validated end-to-end (integration testing) on:

- **Synthetic test images** - a procedurally generated textured scene, split into two overlapping, perspective-warped crops with a known ground-truth relationship.
- **Real camera photographs** - a 3-image sequence of a real waterfront scene (see Section 9), to confirm the pipeline works on unmodified, real-world input, not just controlled synthetic data.

```
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/panorama-stitching
collecting ... collected 4 items
tests/test_homography.py::test_dlt_recovers_known_homography_exactly PASSED [25%]
tests/test_homography.py::test_dlt_raises_on_too_few_points PASSED [50%]
tests/test_homography.py::test_apply_homography_identity PASSED [75%]
tests/test_ransac.py::test_ransac_recovers_homography_with_outliers PASSED [100%]
===== 4 passed in 0.44s =====
```

Figure 10.1 - pytest run: all 4 unit tests passing

10.1 Test Coverage Summary

Layer	Coverage approach
Geometric core (DLT, RANSAC)	Automated pytest unit tests against synthetic ground-truth homographies (see table above); deterministic, fast, and independent of any image data.
Feature detection / matching	Indirectly validated through the end-to-end integration runs; correctness is checked visually (Figures 9.2 and 9.3) rather than with a numeric unit test, since there is no single ground-truth answer for "which keypoints should be detected" in a real photograph.
Warping / canvas sizing	Exercised implicitly by every end-to-end run; verified by checking that the full content of both input images appears in the output canvas with no cropping.
Blending	Verified visually by inspecting the overlap region of the output panoramas (Figures 9.4 and 9.6) for absence of a hard seam line.
CLI / error handling	Manually exercised by running the tool against malformed inputs (an empty folder, a folder with a single image, a folder with non-overlapping images) and confirming a clear error message and non-zero exit code rather than a crash or silent bad output.

10.2 Known Limitations of the Testing Approach

- The feature detection, matching, warping, and blending stages do not have dedicated numeric unit tests, since their "correctness" is inherently visual/perceptual for real photographs; this is a deliberate trade-off, not an oversight, and is compensated for by end-to-end visual verification on both a synthetic scene with a known ground truth and real, unmodified camera photographs.
- Testing was performed on a limited set of scenes (one synthetic textured scene, one 3-photo real waterfront sequence); the pipeline has not been stress-tested on adversarial cases such as very low-texture scenes (e.g. a plain sky) or extreme lighting differences between shots.
- No automated performance/regression benchmarking is included; the runtime figures quoted in Section 4 (NFR1) are single manual measurements on one development machine, not a tracked benchmark suite.

10.3 Independent Reproduction on a Second Machine

To confirm the pipeline is not fragile to a specific development environment, every command in this section was independently re-run on a separate Windows machine (Python 3.14.5, OpenCV 5.0.0, a freshly created virtual environment installed from requirements.txt only), rather than the original Linux development environment. All results matched exactly, including the RANSAC inlier counts and reprojection errors down to the millisecond-timing-independent metrics,

confirming that the fixed RANSAC seed (Section 7.5) makes the pipeline fully deterministic across machines and Python/OpenCV versions.

```
Command Prompt
Location: B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching\venv\Lib\site-packages
Requires:
Required-by: contourpy, matplotlib, opencv-contrib-python, opencv-python

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python -c "import cv2; print(cv2.__version__)"
5.0.0

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python main.py --input data/sample --output data/output/my_panorama.jpg
[INFO] === Panorama Stitching Pipeline ===
[INFO] Loaded img_01.jpg (800x600)
[INFO] Loaded img_02.jpg (900x600)
[INFO] --- Stitching image 2/2 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 423 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 750 keypoints
[INFO] Matching features...
[INFO] Found 80 good matches after ratio test (out of 423 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 40/80 inliers (50.0%)
[INFO] After refinement: 40/80 inliers, mean reprojection error = 0.233px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] Saved result to data/output/my_panorama.jpg
[INFO] Done in 0.95 seconds.

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>
```

Figure 10.2 - Independent run on the 2-image sample dataset (Windows, Python 3.14.5, OpenCV 5.0.0)

```
(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python -m pytest tests/ -v
===== test session starts =====
platform win32 -- Python 3.14.5, pytest-9.1.1, pluggy-1.6.0 -- B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching\venv\Scripts\python.exe
cachedir: .pytest_cache
rootdir: B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching
collected 4 items

tests/test_homography.py::test_dlt_recovers_known_homography_exactly PASSED [ 25%]
tests/test_homography.py::test_dlt_raises_on_too_few_points PASSED [ 50%]
tests/test_homography.py::test_apply_homography_identity PASSED [ 75%]
tests/test_ransac.py::test_ransac_recovers_homography_with_outliers PASSED [100%]

===== 4 passed in 0.31s =====

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>
```

Figure 10.3 - Independent pytest run: all 4 unit tests passing on the second machine

```
===== 4 passed in 0.31s =====

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>python main.py --input docs/input_photos --output data/output/boat_panorama_a_test.jpg
[INFO] === Panorama Stitching Pipeline ===
[INFO] Loaded input_1.jpg (1000x666)
[INFO] Loaded input_2.jpg (1000x666)
[INFO] Loaded input_3.jpg (1000x666)
[INFO] --- Stitching image 2/3 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 1685 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 1235 keypoints
[INFO] Matching features...
[INFO] Found 404 good matches after ratio test (out of 1685 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 380/404 inliers (94.1%)
[INFO] After refinement: 378/404 inliers, mean reprojection error = 0.439px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] --- Stitching image 3/3 onto panorama ---
[INFO] Detecting features in image A...
[INFO] Detected 2158 keypoints
[INFO] Detecting features in image B...
[INFO] Detected 1148 keypoints
[INFO] Matching features...
[INFO] Found 350 good matches after ratio test (out of 2158 candidate pairs)
[INFO] Estimating homography with RANSAC...
[INFO] RANSAC: best sample had 280/350 inliers (80.0%)
[INFO] After refinement: 278/350 inliers, mean reprojection error = 0.547px
[INFO] Computing output canvas...
[INFO] Warping images into shared canvas...
[INFO] Blending...
[INFO] Saved result to data/output/boat_panorama_test.jpg
[INFO] Done in 1.77 seconds.

(venv) B:\Semester-VI\CV\Test\panorama-stitching-complete_Test\panorama-stitching>
```

Figure 10.4 - Independent run on the real 3-photo boat dataset; RANSAC inliers (378/404, 278/350) and reprojection errors (0.439px, 0.547px) exactly match the results reported in Section 9.7

11. Challenges Faced

- **Numerical instability in DLT:** initial attempts to solve the DLT linear system directly on raw pixel coordinates produced inaccurate homographies. Resolved by implementing Hartley point normalization before solving and un-normalizing the result.
- **Outlier contamination in feature matches:** Lowe's ratio test alone still leaves some incorrect matches (visible in Figure 10.2), which corrupt a plain least-squares DLT fit. Resolved by implementing RANSAC on top of DLT to robustly reject outliers.
- **Canvas sizing for warped images:** the warped second image can extend outside the first image's original coordinate frame in any direction. Resolved by projecting all four corners of the second image through the homography, computing a bounding box over both images' corners, and applying a translation homography so all content maps to non-negative coordinates.
- **Visible seams from direct overlay:** simply overlaying warped images at their correct positions produced a visible seam due to exposure/lighting differences between photos. Resolved with distance-transform feathered blending.
- **Choosing the RANSAC iteration count and pixel threshold:** too few iterations occasionally failed to find the true inlier set on the noisier real-photo matches, while too tight a pixel threshold rejected valid inliers that had a small amount of natural reprojection noise. Resolved empirically by testing several values against the real-photo dataset and settling on 2000 iterations and a threshold that kept the reprojection error of the accepted inliers under about 1 pixel on average (Section 9.6).
- **Debugging without a ground truth on real photographs:** unlike the synthetic test, the real waterfront photographs have no known ground-truth homography, so a bug could not be caught by comparing against an exact expected answer. Resolved by relying on visual continuity checks (the horizon line and ship masts staying straight across the stitch boundary) as a practical substitute ground truth.

12. Learnings and Key Takeaways

- Implementing DLT from scratch clarified why point normalization is not an optional detail but a numerical necessity for an SVD-based linear solve on pixel-scale data.
- RANSAC's power comes from a simple statistical argument: a minimal sample drawn entirely from inliers will have far more support from the rest of the data than any sample containing an outlier, given enough random trials.
- Robust estimation (RANSAC) and precise estimation (DLT refit on inliers) are complementary, not interchangeable - RANSAC's 4-point fits are only used to identify the inlier set; the final homography is refit on all inliers for accuracy.
- Visually verifiable outputs (grid-line alignment in the synthetic test, horizon/mast alignment in the real photo test) are an effective way to sanity-check a geometric algorithm's correctness beyond unit tests alone.
- Modular, single-responsibility design paid off directly during debugging: because each pipeline stage is a pure function with a narrow input/output contract, a bug could always be isolated to one module by inspecting its output in isolation, rather than having to reason about the entire pipeline at once.
- Writing the unit tests against synthetic, controllable data before relying on real photographs made it possible to validate the mathematics independently of anything that could go wrong in feature detection or image I/O, which sharply narrowed down where a bug could be.

- Working through this project reinforced that "from scratch" implementations are valuable less because they outperform library functions (they generally do not) and more because they force an explicit, testable understanding of exactly what those library functions are doing internally.

13. Future Enhancements

- Multi-band (Laplacian pyramid) blending for sharper, more artifact-free seams than simple feathering.
- Automatic image ordering/adjacency detection, instead of relying on filename sort order.
- Global bundle adjustment across all images to reduce drift when stitching many images.
- Cylindrical or spherical projection to support full 360-degree panoramas.
- **Exposure compensation:** estimate and correct per-image gain/brightness differences before blending, which would remove the residual brightness seam that pure feathering cannot fully hide under strong exposure mismatch between shots.
- **GPU or vectorized acceleration:** the RANSAC loop's 2000 independent iterations are naturally parallel; moving the inlier-counting step to a vectorized batch operation (or a GPU) would substantially cut runtime for larger images or longer panorama sequences.
- **A minimal graphical front end:** while the CLI satisfies the course's terminal-executability requirement, an optional thin web or desktop preview (drag-and-drop input images, live progress) would make the tool more approachable for non-technical users without changing the underlying pipeline.

Appendix A: Selected Source Code

This appendix reproduces the full source of the two modules that implement the core Module 2 mathematics discussed in Section 8: normalized DLT homography estimation (homography.py) and RANSAC robust estimation (ransac.py). The complete source tree, including every other module and the unit tests, is available in the accompanying GitHub repository.

A.1 homography.py

```
"""
homography.py
-----
Module 3a of the pipeline: HOMOGRAPHY ESTIMATION (Direct Linear Transform).

A homography H is a 3x3 matrix that maps a point (x, y) in image B to
a point (x', y') in image A, in homogeneous coordinates:

    [x'] [h1 h2 h3] [x]
    [y'] ~ [h4 h5 h6] [y]    (~ means "equal up to scale")
    [w'] [h7 h8 h9] [1]

    x' = (h1*x + h2*y + h3) / (h7*x + h8*y + h9)
    y' = (h4*x + h5*y + h6) / (h7*x + h8*y + h9)

DLT (Direct Linear Transform) finds H from >= 4 point correspondences
by rearranging the equations above into a linear system A h = 0 and
solving for h via SVD, exactly as covered in the course notes on
homography estimation.
"""

import numpy as np

def normalize_points(points: np.ndarray):
    """Hartley normalization: shift points so their centroid is the origin
    and scale them so the average distance from the origin is sqrt(2).

    Why: DLT is solved by SVD on a matrix built directly from raw pixel
    coordinates (which can be in the thousands). Without normalization,
    the linear system is numerically ill-conditioned and the solution
    becomes inaccurate. This is a standard, well-known fix (Hartley,
    1997) and is worth explicitly calling out in the report as a design
    decision, since it is easy to skip and quietly get a worse result.

    Returns
    -----
    normalized_points : np.ndarray, shape (N, 2)
    T : np.ndarray, shape (3, 3)
        The similarity transform used, so it can be undone later.
    """
    centroid = points.mean(axis=0)
    shifted = points - centroid
    mean_dist = np.mean(np.sqrt(np.sum(shifted ** 2, axis=1)))
    if mean_dist < 1e-8:
```

```

    mean_dist = 1e-8
    scale = np.sqrt(2) / mean_dist

    T = np.array([
        [scale, 0, -scale * centroid[0]],
        [0, scale, -scale * centroid[1]],
        [0, 0, 1],
    ])

    ones = np.ones((points.shape[0], 1))
    homogeneous = np.hstack([points, ones])
    normalized = (T @ homogeneous.T).T
    return normalized[:, :2], T

```

```

def compute_homography_dlt(points_a: np.ndarray, points_b: np.ndarray) -> np.ndarray:
    """Estimate the homography mapping points_b -> points_a using normalized DLT.

```

Parameters

points_a, points_b : np.ndarray, shape (N, 2), N >= 4
 Matched point coordinates; points_b[i] maps to points_a[i].

Returns

H : np.ndarray, shape (3, 3)
 The estimated homography, normalized so $H[2, 2] == 1$.
 """

```

if points_a.shape[0] < 4 or points_b.shape[0] < 4:
    raise ValueError("Need at least 4 point correspondences to compute a homography")

```

Step 1: normalize both point sets for numerical stability.

```

norm_a, T_a = normalize_points(points_a)
norm_b, T_b = normalize_points(points_b)

```

Step 2: build the $2N \times 9$ linear system $Ah = 0$.

```

n = norm_a.shape[0]
A = np.zeros((2 * n, 9))
for i in range(n):
    x, y = norm_b[i]
    xp, yp = norm_a[i]
    A[2 * i] = [-x, -y, -1, 0, 0, 0, x * xp, y * xp, xp]
    A[2 * i + 1] = [0, 0, 0, -x, -y, -1, x * yp, y * yp, yp]

```

Step 3: solve via SVD. The solution h is the singular vector

corresponding to the smallest singular value of A (last row of V^T).

```

_, _, Vt = np.linalg.svd(A)
h = Vt[-1]
H_norm = h.reshape(3, 3)

```

Step 4: undo the normalization: $H = T_a^{-1} * H_{\text{norm}} * T_b$

```

H = np.linalg.inv(T_a) @ H_norm @ T_b

```

```
# Normalize so the bottom-right element is 1 (standard convention).
if abs(H[2, 2]) > 1e-8:
    H = H / H[2, 2]
```

```
return H
```

```
def apply_homography(H: np.ndarray, points: np.ndarray) -> np.ndarray:
    """Project points through homography H (handles the perspective divide)."""
    ones = np.ones((points.shape[0], 1))
    homogeneous = np.hstack([points, ones])
    projected = (H @ homogeneous.T).T
    # Perspective divide: divide by the homogeneous w-coordinate.
    w = projected[:, 2:3]
    w[w == 0] = 1e-8
    return projected[:, :2] / w
```

```
def reprojection_error(H: np.ndarray, points_a: np.ndarray, points_b: np.ndarray) -> np.ndarray:
    """Per-point Euclidean distance between the true point_a and H applied to point_b.
```

```
    This is the metric RANSAC uses to decide whether a match is an
    inlier (consistent with H) or an outlier.
    """
```

```
    projected = apply_homography(H, points_b)
    return np.sqrt(np.sum((projected - points_a) ** 2, axis=1))
```

A.2 ransac.py

```
"""
```

```
ransac.py
```

```
-----
```

Module 3b of the pipeline: RANSAC (RANDOM Sample Consensus).

Problem: feature_matching.py still leaves some WRONG matches (outliers) even after the ratio test. A single wrong match can badly corrupt a least-squares DLT fit, since DLT tries to satisfy ALL points at once.

RANSAC's idea: repeatedly guess.

1. Randomly pick the minimal number of correspondences needed (4, for a homography).
2. Fit a homography to just those 4 points (cheap, exact).
3. Check how many of the OTHER correspondences agree with this homography (within some pixel-distance threshold) -- these are the "inliers" for this guess.
4. Repeat many times, keep the guess with the most inliers.
5. Finally, refit the homography using ALL of that best guess's inliers (not just 4 points) for a more accurate final result.

Over enough iterations, a sample of 4 correct (inlier) matches is very likely to be drawn at least once, and that sample will have far more support (inliers) than any sample containing an outlier -- which is what lets RANSAC find the correct model despite contamination.

```
"""
```

```
import numpy as np
```

```
from src.homography import compute_homography_dlt, reprojection_error
```

```
from src.utils import log
```

```
def ransac_homography(points_a: np.ndarray, points_b: np.ndarray,  
                      num_iterations: int = 2000,  
                      inlier_threshold: float = 4.0,  
                      min_inliers: int = 10,  
                      seed: int = 42):
```

```
    """Robustly estimate a homography from noisy point correspondences.
```

```
    Parameters
```

```
    -----
```

```
    points_a, points_b : np.ndarray, shape (N, 2)
```

```
        Matched points; points_b[i] maps to points_a[i].
```

```
    num_iterations : int
```

```
        Number of random 4-point samples to try.
```

```
    inlier_threshold : float
```

```
        Max reprojection error (pixels) for a match to count as an inlier.
```

```
    min_inliers : int
```

```
        Minimum inliers required to accept the final model.
```

```
    seed : int
```

```
        RNG seed, for reproducible results (important for grading/demo)
```

reproducibility).

Returns

best_H : np.ndarray, shape (3, 3)

Homography refit on all inliers of the best sample found.

best_inlier_mask : np.ndarray, shape (N,), dtype=bool

Which of the N input matches were inliers to best_H.

"""

rng = np.random.default_rng(seed)

n = points_a.shape[0]

if n < 4:

raise ValueError("Need at least 4 matches to run RANSAC")

best_inlier_count = -1

best_inlier_mask = None

best_H = None

for iteration in range(num_iterations):

Step 1: randomly sample 4 correspondences (the minimum needed).

sample_idx = rng.choice(n, size=4, replace=False)

sample_a = points_a[sample_idx]

sample_b = points_b[sample_idx]

Step 2: fit a homography to just these 4 points.

try:

H_candidate = compute_homography_dlt(sample_a, sample_b)

except np.linalg.LinAlgError:

continue # degenerate sample (e.g. collinear points); skip it

Step 3: count inliers among ALL matches, not just the sample.

errors = reprojection_error(H_candidate, points_a, points_b)

inlier_mask = errors < inlier_threshold

inlier_count = int(np.sum(inlier_mask))

if inlier_count > best_inlier_count:

best_inlier_count = inlier_count

best_inlier_mask = inlier_mask

best_H = H_candidate

if best_H is None or best_inlier_count < min_inliers:

raise RuntimeError(

f"RANSAC failed to find a good homography "

f"(best had {best_inlier_count} inliers, needed >= {min_inliers}). "

f"The images likely do not overlap enough."

)

log(f"RANSAC: best sample had {best_inlier_count}/{n} inliers "

f"({100 * best_inlier_count / n:.1f}%)")

Step 5: refit using ALL inliers of the winning model for a more

accurate final homography (4 points alone are noise-sensitive).

```
        refined_H = compute_homography_dlt(points_a[best_inlier_mask],
points_b[best_inlier_mask])

# Recompute the inlier mask against the refined homography.
final_errors = reprojection_error(refined_H, points_a, points_b)
final_inlier_mask = final_errors < inlier_threshold
log(f'After refinement: {int(np.sum(final_inlier_mask))}/{n} inliers, '
    f'mean reprojection error = {final_errors[final_inlier_mask].mean():.3f}px')

return refined_H, final_inlier_mask
```

A.3 blending.py

```
"""
```

```
blending.py
```

```
-----
```

Module 5 of the pipeline: BLENDING.

Once both images are warped into the same canvas, simply overlaying them produces a visible seam where they overlap (brightness/exposure differences between the two photos become obvious). We fix this with FEATHERING (alpha blending weighted by distance from each image's edge):

- For each warped image, compute a "distance map": every pixel's distance to the nearest black/empty border of that image.
- In the overlap region, blend the two images using weights proportional to each pixel's distance value: pixels deep inside an image (far from its edge) are trusted more than pixels near its edge, where warping artifacts are most visible.
- Outside the overlap, just keep whichever image has valid pixels.

```
"""
```

```
import cv2
```

```
import numpy as np
```

```
def _distance_weight(image: np.ndarray) -> np.ndarray:
```

```
    """Distance transform of the non-black region of `image`, normalized to [0, 1]."""
```

```
    gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

```
    mask = (gray > 0).astype(np.uint8)
```

```
    dist = cv2.distanceTransform(mask, cv2.DIST_L2, 5)
```

```
    max_val = dist.max()
```

```
    if max_val > 0:
```

```
        dist = dist / max_val
```

```
    return dist
```

```
def feather_blend(canvas_a: np.ndarray, canvas_b: np.ndarray) -> np.ndarray:
```

```
    """Blend two same-size warped images using distance-weighted feathering.
```

```
    Both inputs are expected to be on the same canvas: black (0,0,0)
```

```
    pixels represent "no data" from that source.
```

```
    """
```

```
    weight_a = _distance_weight(canvas_a)
```

```
    weight_b = _distance_weight(canvas_b)
```

```
    total_weight = weight_a + weight_b
```

```
    # Avoid division by zero where neither image has data.
```

```
    total_weight_safe = np.where(total_weight == 0, 1, total_weight)
```

```
    norm_a = (weight_a / total_weight_safe)[..., None]
```

```
    norm_b = (weight_b / total_weight_safe)[..., None]
```

```
blended = (canvas_a.astype(np.float64) * norm_a +  
           canvas_b.astype(np.float64) * norm_b)  
  
return np.clip(blended, 0, 255).astype(np.uint8)
```

A.4 stitcher.py

This module ties every other stage together into the end-to-end, multi-image pipeline described in Section 5.

```
"""
```

```
stitcher.py
```

```
-----
```

Module 6: THE STITCHER — orchestrates modules 1-5 into a working pipeline.

Given a list of images (in left-to-right order), stitches them into a single panorama by processing pairs sequentially: stitch image[0] and image[1] into a partial panorama, then stitch that result with image[2], and so on. This keeps the pipeline simple (only ever needs pairwise homography estimation) while still supporting any number of input images.

```
"""
```

```
import numpy as np
from src.feature_detection import detect_features
from src.feature_matching import match_features, get_matched_points
from src.ransac import ransac_homography
from src.warping import compute_canvas, warp_image
from src.blending import feather_blend
from src.utils import log
```

```
def stitch_pair(image_a: np.ndarray, image_b: np.ndarray) -> np.ndarray:
```

```
    """Stitch two overlapping images into one panorama.
```

```
    image_a is treated as the fixed reference frame; image_b is
    detected, matched against image_a, and warped to align with it.
```

```
    """
```

```
    log("Detecting features in image A...")
    kp_a, desc_a = detect_features(image_a)
    log("Detecting features in image B...")
    kp_b, desc_b = detect_features(image_b)
```

```
    log("Matching features...")
    matches = match_features(desc_a, desc_b)
    points_a, points_b = get_matched_points(kp_a, kp_b, matches)
```

```
    log("Estimating homography with RANSAC...")
    # We want H that maps image_b -> image_a's frame, so points_b are
    # the "source" and points_a are the "destination" in our DLT
    # convention (see homography.py's docstring).
    H, inlier_mask = ransac_homography(points_a, points_b)
```

```
    log("Computing output canvas...")
    canvas_size, translation = compute_canvas(image_a, image_b, H)
```

```
    log("Warping images into shared canvas...")
    warped_b = warp_image(image_b, translation @ H, canvas_size)
```

```

warped_a = warp_image(image_a, translation, canvas_size)

log("Blending...")
result = feather_blend(warped_a, warped_b)

return result

def crop_black_borders(image: np.ndarray) -> np.ndarray:
    """Crop away the black border left around the panorama after warping.

    Purely cosmetic, but makes the final output look like a real
    panorama photo rather than a photo on a black canvas.
    """
    gray_sum = image.sum(axis=2)
    rows = np.where(gray_sum.sum(axis=1) > 0)[0]
    cols = np.where(gray_sum.sum(axis=0) > 0)[0]
    if len(rows) == 0 or len(cols) == 0:
        return image
    return image[rows.min():rows.max() + 1, cols.min():cols.max() + 1]

def stitch_images(images: list) -> np.ndarray:
    """Stitch a list of >= 2 images (in order) into one panorama."""
    if len(images) < 2:
        raise ValueError("Need at least 2 images to stitch a panorama")

    panorama = images[0]
    for i in range(1, len(images)):
        log(f"--- Stitching image {i + 1}/{len(images)} onto panorama ---")
        panorama = stitch_pair(panorama, images[i])

    return crop_black_borders(panorama)

```

14. References

- R. Hartley and A. Zisserman, *Multiple View Geometry in Computer Vision*, 2nd Edition, Cambridge University Press, 2004.
- R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer-Verlag London, 2011.
- M. A. Fischler and R. C. Bolles, "Random Sample Consensus: A Paradigm for Model Fitting with Applications to Image Analysis and Automated Cartography," *Communications of the ACM*, 24(6), 1981.
- D. G. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," *International Journal of Computer Vision*, 60(2), 2004.
- OpenCV Documentation - <https://docs.opencv.org/>
- NumPy Documentation, [linalg.svd](https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html) - <https://numpy.org/doc/stable/reference/generated/numpy.linalg.svd.html>
- R. I. Hartley, "In Defense of the Eight-Point Algorithm," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 19(6), 1997 (source of the point-normalization technique used in Section 7.2).
- CSE3010 Computer Vision course syllabus, Module 2: Depth Estimation and Multi-Camera Views.
- pytest Documentation - <https://docs.pytest.org/>